

1510: Stone Game IV

Alice and Bob take turns playing a game, with Alice starting first.

Initially, there are n stones in a pile. On each player's turn, that player makes a *move* consisting of removing **any** non-zero **square number** of stones in the pile.

Also, if a player cannot make a move, he/she loses the game.

Given a positive integer n , return `true` if and only if Alice wins the game otherwise return `false`, assuming both players play optimally.

Example 1:

Input: $n = 1$

Output: `true`

Explanation: Alice can remove 1 stone winning the game because Bob doesn't have any moves.

Example 2:

Input: $n = 2$

Output: `false`

Explanation: Alice can only remove 1 stone, after that Bob removes the last one winning the game (2 -> 1 -> 0).

Example 3:

Input: $n = 4$

Output: `true`

Explanation: n is already a perfect square, Alice can win with one move, removing 4 stones (4 -> 0).

Constraints:

- $1 \leq n \leq 10^5$

Dynamic Programming

Intuition

Since both players play optimally, Zermelo's theorem guarantees that every state is either winning or losing for the current player.

- The current player **wins** if at least a **single move** leaves the opponent in a **losing** position.
-

Alice is the **starting player**, beginning the game with **n** stones.

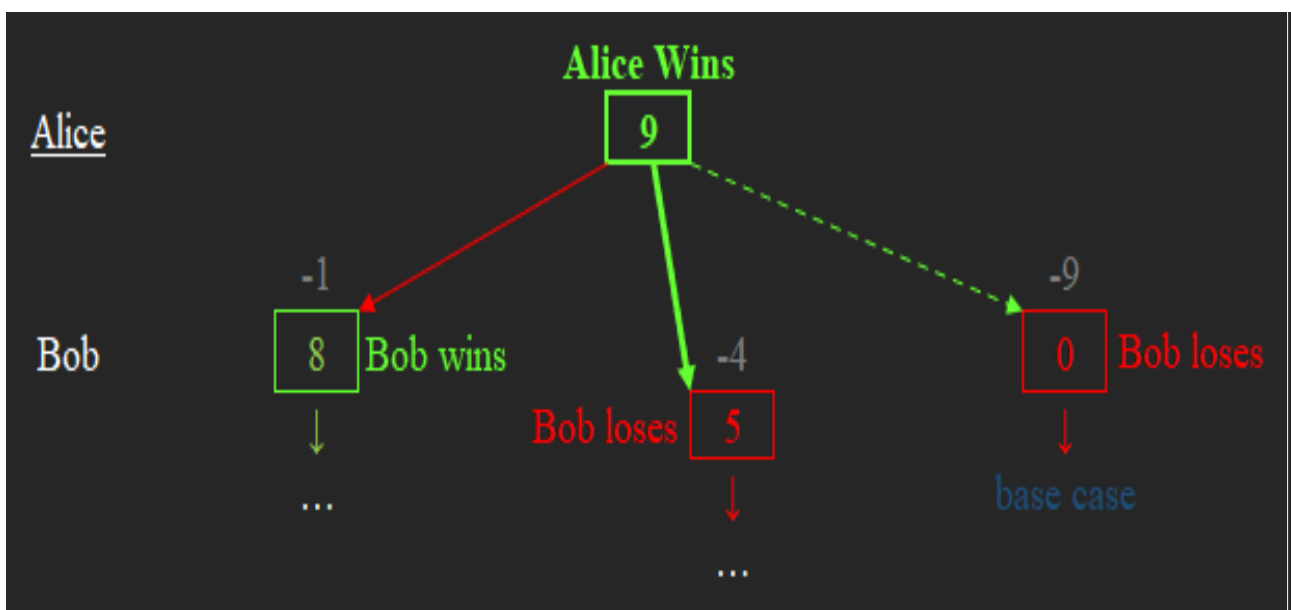
She can remove any **square number** of stones:

1, 4, 9, 16, ...

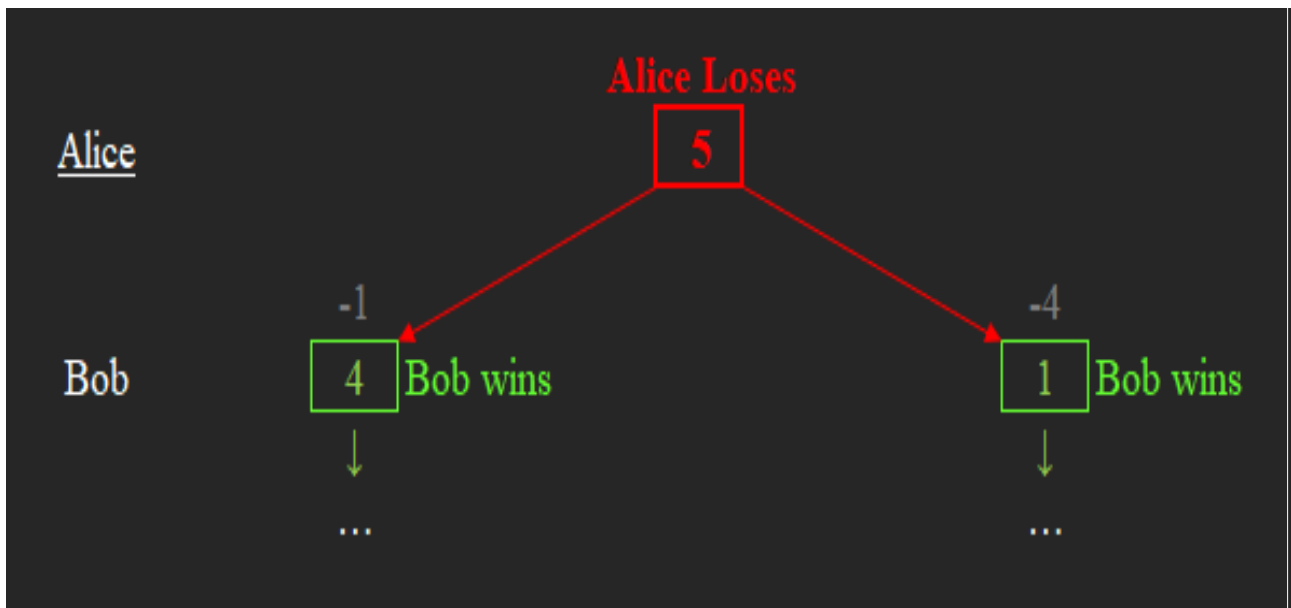
To simulate the game, we can think of it recursively:

After Alice removes either **1**, or **4**, or **9...** stones, the game moves to a **smaller number** of remaining stones:

- **Case 1:** If any of the choices **recursively** leaves Bob in a **losing** position, Alice can select this move and **win**.



- **Case 2:** Otherwise, if **every** possible choice **recursively** leaves Bob in a **winning** position, Alice is guaranteed to **lose**.



Some states may be reached recursively multiple times.

We cache each computed state so that once its result is known, we can reuse it instead of recomputing it.

Python:

```
class Solution:
    def winnerSquareGame(self, n: int) -> bool:
        @cache
        def dfs(i):
            if i == 0: return False

            for j in range(1, isqrt(i) + 1):
                if not dfs(i - j ** 2): return True

            return False

        return dfs(n)
```

JavaScript:

```
const dfs = _memoize(i => { if (!i) return 0; for (let j = 1; (j ** 2) <= i; j++) if (!dfs(i - j ** 2)) return 1; return 0; });
const winnerSquareGame = n => dfs(n);
```

The recursion is easy to understand, but The DP (tabulation) is a lot more straightforward:

Since every state is determined only by the states reached after removing a square number, we can simply compute the `states` from left to right.

We mark every state as `winning`, if `at least one` reachable state is `losing`:

`dp[i]=true` IF `dp[i-12]=false`,

or `dp[i-22]=false`,

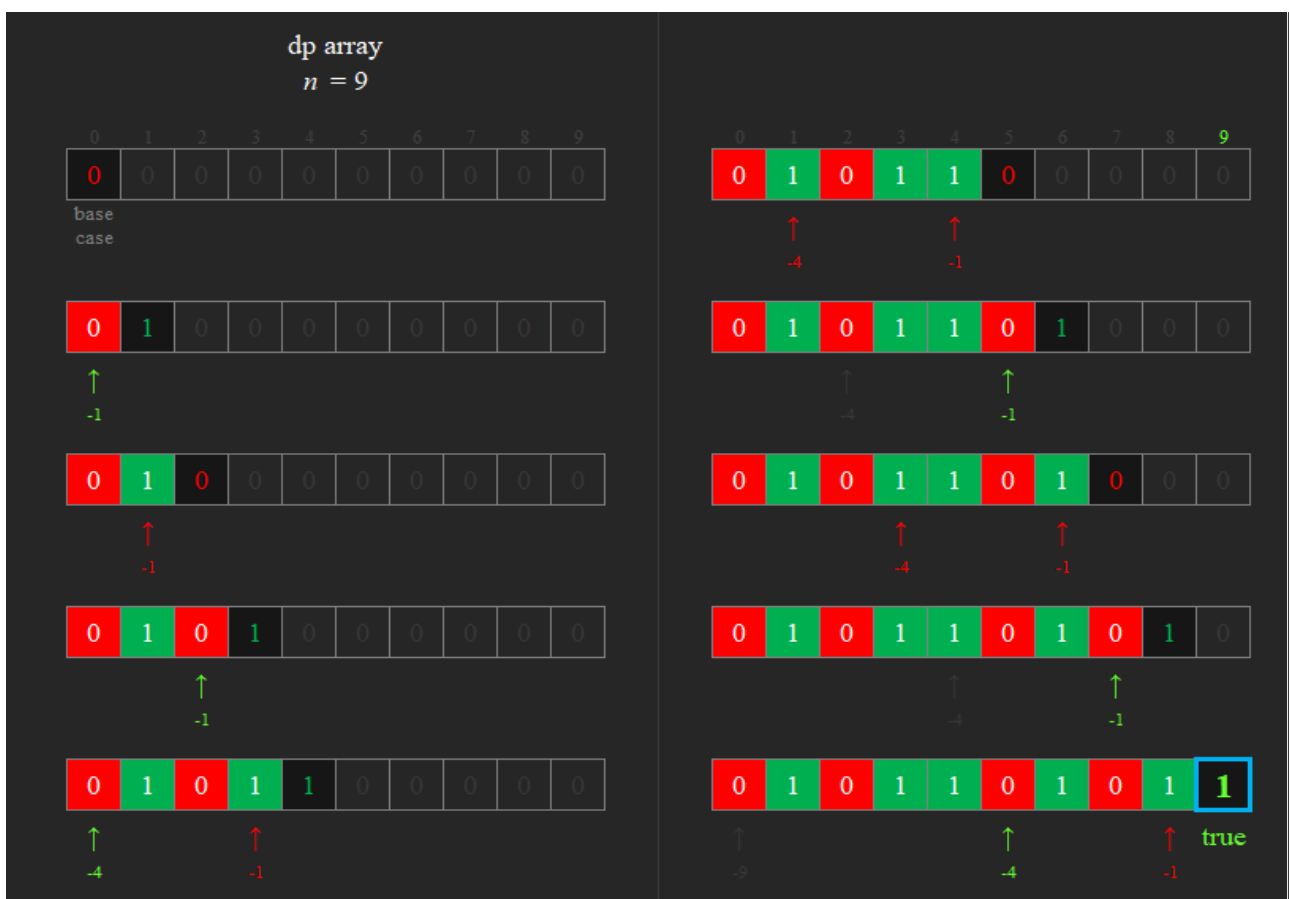
...

or `dp[i-j2]=false`

where: $j \leq i$.

Otherwise, the state is `losing`, `dp[i]=false`.

For example, we want to find the solution for `n=9`:



When we reach `dp[9]`:

- The state `dp[5]` is already known to be losing.
- Therefore, `dp[9]` is `winning`, and the answer for `n=9` is `true`.

Global DP

Since each test case only depends on the value of `n`, all test cases can share the same `dp` array.

Therefore, we can `precompute` the `dp` array once starting at the `max` constraint, and reuse it for every test case.

```
class Solution {  
  
    static final int MAX = 100000;  
  
    static final boolean[] dp = new boolean[MAX + 1];  
  
    static {  
        for (int i = 0; i <= MAX; i++) {  
            if (dp[i]) continue;  
  
            for (int j = 1; j * j <= MAX - i; j++)  
                dp[i + j * j] = true;  
        }  
    }  
  
    public boolean winnerSquareGame(int n) { return dp[n]; }  
}
```

Time Complexity: $O(n) \rightarrow O(1)$ look up

Space Complexity: $O(1)$